N2SCFTDB

$\mathcal{N} = 2$ theories

Mathematical background and implementation reference

One reference, organized by package

Exact Lie-algebra data, anomaly checks, superconformal indices, Coulomb-branch indices and spectra, theory properties, database persistence, and the planned Higgs-branch extension.

Property and database update: two separate stages

Anomaly-checked imports now store basic properties first. A separate worker fills indices and Coulomb spectra, or upgrades each index only at a higher recorded cutoff. Schema 7 preserves legacy results with unknown precision. The index/cache algorithms and dated benchmarks remain documented alongside the 238-test database implementation validation.

Snapshot	Base reference: 8 September 2026. Database and validation update: 9 September 2026. Index/cache update: 10 September 2026; `04bbe21`. Property/database workflow: 12 September 2026.
Consolidates	`index/n2_theory_index_Mathematical_Background.pdf` and the previous `output/pdf/n2_implementation_reference_summary.pdf`.
Status	Basic properties separated from index calculation; schema 7, deferred worker, independent precision upgrades and legacy handling documented. Unrelated mathematics and benchmark chapters retain their dated snapshots. Higgs/Hall–Littlewood material is explicitly marked as proposed, not implemented.
Editing scope	Documentation only. The companion index-background PDF is also updated. Unrevised mathematical and package content is preserved.

1 Project map and conventions

Package	Responsibility	Read here
`anomalies`	Cartan types, representations, full/half hypers, beta functions, local and conventional mod-two gauge anomalies.	§2
`index`	FORM expansion, LiE partial decompositions and SQLite singlet cache; Coulomb PE/PL and full-index projection; future Higgs/flavor refinement.	§3
`common`	Property orchestration, flavor blocks, central charges, exact-number and FORM helpers, JSON and MySQL persistence.	§4
`test`	Unit and backend regression tests, precision and serialization checks, optional live database integration.	§5

How data moves

Theory dictionary / JSON $\longrightarrow$ validated gauge and hyper data
$\longrightarrow$ anomaly and conformality checks
$\longrightarrow$ basic properties and optional MySQL insertion
$\longrightarrow$ separate index / Coulomb-spectrum worker
$\longrightarrow$ fill missing data or upgrade known lower cutoffs

Standalone index functions are also callable directly. The property and database workflows impose the SCFT-candidate checks; a gauge-invariant counting calculation alone is not a proof that an input defines an SCFT.

Notation used throughout

$G = \prod_{a=1}^{s} G_a$	Compact, simply connected, semisimple gauge group; no abelian gauge factor or global quotient is implemented.
$\mathcal{R}_h = \boxtimes_a R_{ha}$	One product-group hyper representation; n_h is its number of copies. Omitted factors are singlets.
$\epsilon_h = 1, \frac{1}{2}$	Weight of a full or half hyper, respectively. Beta functions use $2\epsilon_h T$, not $\epsilon_h T$.
$E, R, r; j_1, j_2$	Scaling dimension, $SU(2)_R$ weight, $U(1)_r$ charge, and Lorentz Cartan weights in the chosen right-index convention.
t, y, u	Project full-index fugacities; the common literature variable $v = u^2$.
x, τ, q	Coulomb grading, proposed HL grading, and an internal FORM integer-power variable. This q is not a conventional full-index fugacity.

What "spectrum" means here

The Coulomb spectrum is the multiset of dimensions of independent chiral-ring generators, with repetitions. Index coefficients count all graded ring elements, including products. Neither is the spectrum of massive BPS particles or the full operator spectrum of the SCFT.

2 Package `anomalies`

2.1 Root data and representation invariants

MODULE `anomalies/lie_algebra.py`

`SimpleLieAlgebra` collects Sage root, weight and character objects together with the rank, dimension, dual Coxeter number and adjoint Dynkin labels. The cached constructor is `get_lie_algebra`. The accepted finite types are A_r $(r \geq 1)$, B_r, C_r $(r \geq 2)$, D_r $(r \geq 4)$, $E_{6,7,8}$, F_4, and G_2. Use A_1 for B_1/C_1, A_3 for D_3, and $A_1 \times A_1$ for D_2.

For a dominant integral highest weight λ, Dynkin labels and dimensions are

$$\lambda = \sum_i a_i \omega_i, \qquad a_i = \langle \lambda, \alpha_i^\vee \rangle \in \mathbb{Z}_{\geq 0}, \tag{2.1}$$

$$\dim V_\lambda = \prod_{\alpha>0} \frac{(\lambda+\rho,\alpha)}{(\rho,\alpha)}, \qquad \rho = \frac{1}{2}\sum_{\alpha>0}\alpha. \tag{2.2}$$

The algebra dimension is $\dim \mathfrak{g} = \operatorname{rank}\mathfrak{g} + |\Phi|$, and the adjoint highest weight is the highest root. `representation_dimension` uses the degree of the Sage Weyl character, implementing (2.2). Source: Sage root-system and Weyl-character documentation [1, 2].

To compensate for the ambient root-length convention used by Sage, the code uses

$$C_2(\lambda) = \frac{(\lambda, \lambda+2\rho)}{\ell^2_{\text{long}}}, \qquad T(R) = \frac{\dim R}{\dim \mathfrak{g}} C_2(R). \tag{2.3}$$

If long roots have squared length two, the first denominator is two. Thus $C_2(\text{adj}) = T(\text{adj}) = h^\vee$ and $T(\mathbf{N}) = 1/2$ for $SU(N)$. The routines are `quadratic_casimir` and `dynkin_index`. The physics normalization is fixed by $\operatorname{Tr}_R(T^a T^b) = T(R)\delta^{ab}$.

Names, conjugation and reality

`named_representation_labels` supports common names such as fundamental, antifundamental, adjoint, singlet, classical spinors and selected two-index representations. Explicit Dynkin labels are the general interface. Numbering is Sage's finite-Cartan numbering; do not copy a label tuple from another reference without checking its node convention.

$$\lambda^* = -w_0\lambda, \qquad \nu(R) = \int_G \chi_R(g^2)\, d\mu(g) \in \{+1, 0, -1\}. \tag{2.4}$$

Here $\nu = +1, 0, -1$ means real, complex, pseudoreal. `conjugate_dynkin_labels` uses the dual character; `representation_reality` uses its Frobenius–Schur indicator. For an irreducible external tensor product, $\nu(\boxtimes_a R_a) = \prod_a \nu(R_a)$. Source: Sage Weyl characters [2]; representation and index conventions can be cross-checked with LieART 2.0 [4].

2.2 Theory data, hypermultiplets and the beta function

MODULE `anomalies/check_n2_anomalies.py`

The general input parser constructs `GaugeFactorData(factor_id,algebra)` objects and simple/product hyper data. Counts are exact nonnegative integers. Malformed types, labels or representations are rejected before calculation. The theory dictionary supports either one simple factor or a list of named gauge factors with a representation for each charged factor.

In $\mathcal{N}=1$ language, a full hyper in $\mathcal{R}$ contains chirals in $\mathcal{R}\oplus\overline{\mathcal{R}}$. A half hyper contains one chiral and is allowed only when the *entire* gauge representation is pseudoreal. Consequently $\mathbf{2}\boxtimes\mathbf{2}$ of $SU(2)^2$ is real and cannot be a single half hyper, whereas $\mathbf{2}\boxtimes\mathbf{2}\boxtimes\mathbf{2}$ is pseudoreal and can. This is validated rather than inferred from one factor alone. Source: Bhardwaj–Tachikawa, §2, Eqs. (2.1)–(2.4) [3].

For factor a, spectator dimensions multiply the effective number of charged degrees of freedom. In the normalization of (2.3),

$$b_{0,a} = 2h_a^\vee - \sum_h n_h\,(2\epsilon_h)\,T_a(R_{ha})\prod_{b\neq a}\dim R_{hb}. \tag{2.5}$$

The implementation uses the literal coefficient two for full hypers and one for half hypers. $b_{0,a}=0$ for every simple factor is its one-loop conformality condition; $b_{0,a}>0$ denotes asymptotic freedom in this convention. Source: Bhardwaj–Tachikawa, Eqs. (2.5)–(2.8) [3].

Checks that expose normalization mistakes

For $SU(N)$ with N_f full fundamental hypers,

$$b_0 = 2N - N_f. \tag{2.6}$$

For one full bifundamental $(\mathbf{N},\mathbf{M})$ of $SU(N)\times SU(M)$, its contributions to the two beta-function subtractions are M and N. For a half hyper in an $SU(2)$ fundamental, the subtraction is $1/2$. These follow directly from (2.5), not from a new convention for T.

> **Two different uses of "index"**
>
> The Dynkin index $T(R)$ is a quadratic trace invariant. It is unrelated to the superconformal index $\mathcal{I}$. LieART's commonly tabulated quadratic index $\ell(R)$ in the old project comparison is $2T(R)$; convert the convention before comparing tables. Root/node numbering must be converted independently.

Scope of the candidate flag

The anomaly checker combines valid matter, gauge-anomaly cancellation and vanishing beta functions into a `lagrangian_scft_candidate` result. It does not reconstruct a Seiberg–Witten geometry, identify S-dual theories, classify line operators, or handle non-Lagrangian matter sectors.

2.3 Local and conventional global gauge anomalies

For a left-handed Weyl fermion in R, the local cubic anomaly tensor is

$$d_R^{abc} = \mathrm{Tr}_R\Big(T^a\{T^b, T^c\}\Big), \qquad d_{\overline{R}}^{abc} = -d_R^{abc}. \tag{2.7}$$

Full hypers cancel between R and $\overline{R}$; self-conjugate real or pseudoreal representations have zero cubic tensor. The adjoint gauginos are in a real representation. Thus local anomaly cancellation follows structurally for the allowed matter schema; the checker need not tabulate every cubic invariant. For semisimple product groups, mixed tensors with one generator from another factor vanish by $\mathrm{Tr}\, T^a = 0$. Source: Bilal, discussion of chiral fermions, conjugate representations and anomaly traces [5]; matter restrictions in [3].

The mod-two test

For factors with the conventional four-dimensional Witten anomaly, the number of anomalous Weyl representations must be even. The code checks A_1, all supported C_r, and $B_2 \simeq C_2$. In its normalization the parity is

$$\mathfrak{a}_a = \sum_{h\ \mathrm{half}} n_h\,[2T_a(R_{ha})] \prod_{b\neq a} \dim R_{hb} \pmod 2. \tag{2.8}$$

Full hypers contribute an even number automatically. The integer entering this test is $2T$, not the full-hyper beta contribution with an additional factor of two. A nonzero parity makes the gauge theory inconsistent within this setup. Source: Witten [6]; Wang–Wen–Witten, §2.3 [7]; symplectic-group discussion in [3, 8].

For $SU(2)$ isospin j, a useful independent formula is

$$T_j = \frac{1}{3}j(j+1)(2j+1), \qquad 2T_j = \frac{2}{3}j(j+1)(2j+1). \tag{2.9}$$

It is odd for $j = 2k + 1/2$, $k \in \mathbb{Z}_{\geq 0}$: $j = 1/2, 5/2, 9/2, \ldots$. One fundamental half hyper is anomalous, two are safe. A single $SU(2)^3$ trifundamental half hyper gives four doublets to each factor and is safe under this conventional test, although it is not by itself conformal.

> **Not an all-background anomaly classification**
>
> The project assumes the ordinary spin-theory setting and simply connected groups. It does not implement the newer non-spin $SU(2)$ anomaly associated with isospin $4k + 3/2$, global quotients, discrete theta angles or general bordism anomalies. Flavor 't Hooft anomalies need not vanish and are not gauge-anomaly rejection criteria. See [7, 8] for the distinction.

2.4 Independent $SU(N)$ formula checks

This section retains the useful representation-theory cross-checks from the older reference. They are derived checks, not a second implementation backend. For Dynkin labels $(a_1, \ldots, a_{N-1})$, define row lengths and box count by

$$\ell_i = \sum_{k=i}^{N-1} a_k \quad (i < N), \qquad \ell_N = 0, \qquad B = \sum_{i=1}^{N} \ell_i. \tag{2.10}$$

Specializing the Weyl dimension and highest-weight Casimir formulas gives

$$\dim R = \prod_{1 \le i < j \le N} \frac{\ell_i - \ell_j + j - i}{j - i}, \tag{2.11}$$

$$C_2(R) = \frac{1}{2}\left[NB + \sum_{i=1}^{N} \ell_i(\ell_i + 1 - 2i) - \frac{B^2}{N} \right], \tag{2.12}$$

$$T(R) = \frac{\dim R}{N^2 - 1} C_2(R). \tag{2.13}$$

Source: Direct specialization of (2.2) and (2.3); compare Sage characters [2] and representation tables in [4].

$SU(N)$ representation	Dimension	C_2	T
Fundamental	N	$(N^2-1)/(2N)$	$1/2$
Adjoint	N^2-1	N	N
Two-index symmetric	$N(N+1)/2$	$(N-1)(N+2)/N$	$(N+2)/2$
Two-index antisymmetric	$N(N-1)/2$	$(N+1)(N-2)/N$	$(N-2)/2$

One symmetric plus one antisymmetric full hyper therefore gives $2T_{\text{sym}} + 2T_{\text{asym}} = 2N$ and $b_0 = 0$. For $N = 2$ the antisymmetric is a singlet, with zero T.

Function-to-equation map

Routine	Mathematical object
`get_lie_algebra`, `validate_dynkin_labels`	Root data and (2.1)
`representation_dimension`	(2.2), checked by (2.11)
`quadratic_casimir`, `dynkin_index`	(2.3)
`conjugate_dynkin_labels`, `representation_reality`	(2.4)
Gauge-factor beta calculation	(2.5)
Gauge-anomaly checks	(2.7), (2.8)

3 Package `index`

3.1 Full-index convention and single-letter functions

MODULE `index/n2_theory_index.py`

In radial quantization the chosen right-handed supercharge restricts the trace to

$$\delta_R = E - 2j_2 - 2R + r = 0, \tag{3.1}$$

$$\mathcal{I}(t,y,u) = \mathrm{Tr}_{\delta_R=0}(-1)^F t^{2(E+j_2)} y^{2j_1} u^{-2(r+R)}. \tag{3.2}$$

This is the literature convention with $v = u^2$. It is a signed protected trace, not a positive count of every short multiplet: cancellations and recombination mean the full operator spectrum cannot generally be recovered uniquely. Source: Gadde–Pomoni–Rastelli, Eqs. (5.4)–(5.5) [9].

Writing $D(t,y) = (1-t^3y)(1-t^3/y)$, the scalar coefficients of the vector and half-hyper characters are

$$f_V(t,y,u) = \frac{t^2u^2 - t^4u^{-2} - t^3(y+y^{-1}) + 2t^6}{D(t,y)}, \tag{3.3}$$

$$f_H(t,y,u) = \frac{t^2/u - t^4u}{D(t,y)}. \tag{3.4}$$

The denominator counts the two allowed spacetime derivatives; fermionic letters and equations of motion produce the numerator signs. Source: Gadde–Pomoni–Rastelli, Eqs. (5.21)–(5.22), with $v = u^2$ [9].

For gauge variables $g = (g_1, \ldots, g_s)$, the complete letter function is

$$L = f_V \sum_a \chi_{\mathrm{adj}_a}(g_a) + f_H \left[\sum_{h\ \mathrm{full}} n_h(\chi_{\mathcal{R}_h} + \chi_{\overline{\mathcal{R}_h}}) + \sum_{h\ \mathrm{half}} n_h \chi_{\mathcal{R}_h} \right]. \tag{3.5}$$

Here $\chi_{\mathcal{R}_h}(g) = \prod_a \chi_{R_{ha}}(g_a)$. The conjugate of a product representation conjugates every factor simultaneously: $(R,S) \mapsto (\overline{R},\overline{S})$. A full bifundamental is not four independent choices of conjugation. For a self-conjugate full hyper, the two equal characters still give multiplicity two.

> **Flavor-unrefined does not mean flavor-singlet**
>
> All flavor fugacities are currently set to one. Flavor representations are replaced by their dimensions, while gauge representations are kept until singlet projection. Charged flavor operators are counted; the code does not project them out. Spin and R-symmetry fugacities y, u are still present.

3.2 Plethystic expansion and FORM truncation

For a letter function with no degree-zero term, define

$$\mathrm{PE}[L] = \exp\left[\sum_{k\geq 1} \frac{1}{k} L(t^k, y^k, u^k; g_1^k, \ldots, g_s^k)\right], \qquad \mathcal{I} = \int_G d\mu(g)\ \mathrm{PE}[L]. \tag{3.6}$$

Adams substitution acts on every fugacity and character, but leaves numerical multiplicities and the signed letter coefficients as coefficients. The Haar measure is normalized so $\int_G 1 = 1$. Source: Plethystics [10]; gauge-index construction in [9].

`_character_basis` assigns formal character functions $C_p(k) = \chi_{R_p}(g_a^k)$ to simple-factor irreps. `_build_form_program` constructs the truncated exponent

$$A_D = \left[\sum_{k=1}^{\lfloor D/2\rfloor} \frac{L(t^k, y^k, u^k; g^k)}{k}\right]_{t\leq D}. \tag{3.7}$$

Every letter begins at t^2, so $k > \lfloor D/2\rfloor$ and particle number $m > \lfloor D/2\rfloor$ cannot contribute. Derivatives are expanded as

$$D(t,y)^{-1} = \sum_{p,q\geq 0} t^{3(p+q)} y^{p-q}; \qquad p, q \leq \lfloor D/3\rfloor \tag{3.8}$$

is a sufficient rectangular bound before final degree truncation. FORM declares t with maximum power D, so excess combined powers are discarded throughout the calculation. Negative powers of u and y are allowed.

The auxiliary-variable recurrence

The code starts with zA_D and successively substitutes

$$z \longmapsto 1 + \frac{zA_D}{m}, \qquad m = 2, \ldots, M, \quad M = \lfloor D/2\rfloor. \tag{3.9}$$

After step m, the expression is $\sum_{j=1}^{m-1} A_D^j/j! + zA_D^m/m!$. Setting $z = 1$ and adding the vacuum term gives $\sum_{j=0}^{M} A_D^j/j!$ exactly through degree D. This is the algebraic reason for the generated FORM substitutions, rather than an approximation requiring a convergence assumption. Source: Recurrence derived from the source; FORM syntax and truncation conventions in the supplied FORM 5 manual [12], with software background in [13].

`run_form` is imported from `common/form_utils.py`. `FormExpansionCache.parse_form_output` in `index/form_expansion_cache.py` converts rational coefficients, fugacity exponents and character powers into `IndexFormTerm` records. `get_expansion` reuses their SQLite serialization when the complete raw FORM program is already cached. The numerical coefficients remain exact rationals, including cancellations generated by the index.

3.3 Gauge singlets, LiE and the reusable character cache

MODULE `index/char_decomposition_cache.py`

The Adams operation is

$$\psi^k \chi_R(g) = \chi_R(g^k), \qquad \prod_{k\geq 1} (\psi^k \chi_R)^{n_k} = \sum_\lambda c_\lambda \chi_\lambda. \tag{3.10}$$

The right-hand side may be a *virtual* character: c_λ need not be nonnegative. For $SU(2)$, $\psi^2 \chi_{\mathbf{2}} = \chi_{\mathbf{3}} - \chi_{\mathbf{1}}$, while $\chi_{\mathbf{2}}^2 = \chi_{\mathbf{3}} + \chi_{\mathbf{1}}$. Replacing the former with the latter would change the PE and corrupt gauge projection. Source: LiE manual, character operations [11]; Sage character operations [2].

A full-decomposition request identifies the Cartan type, Dynkin labels and the complete Adams-power tuple. Its weighted order is

$$w = \sum_{k\geq 1} k n_k. \tag{3.11}$$

This is metadata, not a unique key: $(2,0)$ and $(0,1)$ have the same order but describe different virtual characters. Full decompositions remain available through `get_decomposition`; singlet requests use the smaller calculation below.

Normalized Haar integration and the product-group rule are unchanged:

$$\int_{G_a} \chi_\lambda(g_a)\, d\mu_a = \delta_{\lambda,0}, \qquad \mathrm{mult}_{\mathbf{1},G}(\boxtimes_a V_a) = \prod_a \mathrm{mult}_{\mathbf{1},G_a}(V_a). \tag{3.12}$$

`_project_terms` groups formal products by simple gauge factor and requests one integer per distinct product. It multiplies these integers into the FORM coefficient. Empty products contribute one. B_r, D_r continue to mean Spin representations, not a restriction to representations descending to SO quotients.

Why two partial decompositions suffice

For irreducibles, Schur orthogonality and duality imply

$$\int_G \chi_\lambda(g)\overline{\chi_\mu(g)}\, d\mu(g) = \delta_{\lambda\mu}, \qquad \chi_{\mu^*}(g) = \overline{\chi_\mu(g)},$$
$$[\mathbf{1}](AB) = \sum_\lambda a_\lambda b_{\lambda^*}, \quad A = \sum_\lambda a_\lambda \chi_\lambda, \quad B = \sum_\mu b_\mu \chi_\mu.$$

Equivalently, $(V \otimes W)^G \simeq \mathrm{Hom}_G(V^*, W)$; Hom_G is the vector space of linear maps commuting with the group action. Schur's lemma gives one invariant for dual irreps and zero otherwise. Linearity extends the pairing to signed virtual characters. Source: Etingof, Corollary 35.9 and Proposition 32.1 [22]; Sage `inner_product()` and `dual()` [2]. The displayed contraction follows by substituting the character expansions..

Projection helpers: inputs, outputs and control flow

Helper	Role in the revised algorithm
`_dual_permutation(algebra)`	Cache the permutation of fundamental weights induced by $-w_0$. Apply it to general labels without constructing a new Sage character for each output irrep.
`_singlet_pair(algebra,left,right)`	Return $\sum a_\lambda b_{\lambda^*}$ by looking up dual labels; iterate over the smaller dictionary. Zero and scalar-trivial factors have direct shortcuts.
`_split_character_product`	Expand powers into individual Adams factors, then greedily balance two symbolic partial products.
`_character_singlets`	Plan needed partial decompositions, prefetch same-base-irrep products, expand mixed parts and contract the two sides.
`_tensor_decompositions`	Ask LiE for a full *intermediate* tensor product; never perform the final pairing this way.
`_compute_singlet_multiplicities`	Handle already-decomposed inputs: sort by dictionary size, recursively build two parts, and pair dual irreps.

An input factor `(labels,(n1,n2,...))` means $\prod_j \psi^j(R)^{n_j}$. The splitter assigns each individual $\psi^j(R)$ the estimated cost $j \max(1, \sum_i \lambda_i)$. It sorts largest first and puts the next factor on the lighter side; a tie favors an existing copy of the same base irrep. Repeated factors may separate, while a single Adams operation stays intact. This is a project heuristic, not an optimality theorem.

Inside `_character_singlets`, `plan()` recursively records the required mixed splits and collects requests involving only one base irrep. The existing `get_decompositions()` cache and process pool supply these requests. `expand()` then builds mixed parts with LiE and reuses them in the thread-local `partial_products` dictionary. Only the complete product's final integer is persisted in the singlet table; mixed partial decompositions are memory-only. Trivial base irreps are removed from the calculation because $\psi^j(\mathbf{1}) = \mathbf{1}$.

> **Duality is not conjugate canonicalization**
>
> For A_2, $(a,b)^* = (b,a)$; for A_r, the label order reverses. The general permutation is obtained from the existing Sage duality helper and cached. Actual conjugate orientations remain distinct in character keys. For example, $[\mathbf{1}](\mathbf{3} \otimes \overline{\mathbf{3}}) = 1$, whereas $[\mathbf{1}](\mathbf{3} \otimes \mathbf{3}) = 0$ for $SU(3)$. A pseudoreal irrep pairs with itself with coefficient $+1$; its Frobenius–Schur sign is not inserted into this tensor pairing.

For four $SU(2)$ doublets, decompose the two pairs as $\mathbf{2} \otimes \mathbf{2} = \mathbf{1} \oplus \mathbf{3}$. The final singlet is $1 \cdot 1 + 1 \cdot 1 = 2$, without decomposing the complete four-factor product. The final lookup costs $O(r \min(N_A, N_B))$ label operations for rank r and support sizes N_A, N_B; constructing the partial decompositions can still dominate.

SQLite schema, exact keys and cache lifetime

The character and singlet tables use `sqlite3` and default to `<project-root>/char_decomposition_cache.db`. The independent cache schema has `PRAGMAuser_version=1`; it is unrelated to MySQL theory schema version 7.

Table	Columns and primary key
`character_decompositions`	`algebra TEXT`, `dynkin_labels TEXT`, `adams_powers TEXT`, `adams_order INTEGER`, `terms_json TEXT`. Primary key: `(algebra,dynkin_labels,adams_powers)`.
`singlet_coefficients`	`algebra TEXT`, `product_key TEXT`, `coefficient TEXT`. Primary key: `(algebra,product_key)`.

All columns are `NOT NULL`; both tables use `WITHOUT ROWID`. Dynkin labels and canonical powers are compact JSON text. Coefficients are signed decimal strings, preserving integers beyond SQLite's 64-bit integer storage range [24]. A decomposition payload is a list of `[labels,"coefficient"]` pairs, not a separate JSON cache file.

`_canonical_product` merges repeated labels by adding Adams exponents and sorts factors in descending label order. A symbolic key has the shape `["characters",canonical_product]`. Already-decomposed inputs use a distinct `["decompositions",sorted_encoded_factors]` namespace. The algebra is always part of the SQL key. Product gauge groups reuse simple-factor entries.

`get_singlet_multiplicities()` checks memory and then SQLite before requesting any decomposition. On a miss it invokes `_character_singlets()`, saves the integer and returns results in input order. Cached zero and negative values are valid hits. `get_decomposition()` returns a copy of a stored dictionary; `get_decompositions()` batches missing prerequisites by tensor-factor count. Their full-decomposition algorithm remains available to explicit callers.

Concurrency and failure boundaries

Each thread owns its connection and memory caches. A PID check avoids using an inherited connection; the process-pool parent closes its connection before starting workers, then writes completed results by dependency level. Sequential batches flush at 64 entries and at completion. LiE calculations do not run inside write transactions. Failed calculations do not insert a guessed zero.

WAL permits readers alongside a writer, but still serializes writers; it assumes a local filesystem [23]. The code sets a 30-second busy timeout and retries busy/locked operations. Python's connection context manager controls commit/rollback; this class's `close()` and context manager also close the connection and clear local caches [25].

`cache_directory=` selects a directory for the default filename; `database_path=` selects an explicit file. They are mutually exclusive. The CLI exposes `--cache-directory` and `--cache-database`. Legacy JSON discovery/import and `cache_path()` have been removed. Existing SQLite entries and keys remain valid; the projection update needs no migration.

Persisting the FORM expansion before projection

`index/form_expansion_cache.py` now owns `IndexFormTerm`, the exact parser, and `FormExpansionCache`. For raw program text P, the stored value is the parsed list of monomials before any gauge projection:

$$P \longmapsto \{(c, \alpha, \beta, \gamma, ((p, \mathbf{n}_p))_p)\}\,, \qquad c\, t^{\alpha} y^{\beta} u^{\gamma} \prod_{p,j} C_p(j)^{n_{p,j}}.$$

This notation is a direct description of the project record format. Each `IndexFormTerm` has a `Fraction` coefficient, integer fugacity powers, and a tuple of character indices and Adams-power tuples. The compact JSON row is `["numerator","denominator",t_power,y_power,u_power,characters]`. `_encode_expansion` stores a list of these rows; `_decode_expansion` restores exact arbitrary-size fractions and nested tuples. No float conversion or representation decomposition occurs in this layer.

```
CREATE TABLE form_expansions (
    program TEXT COLLATE BINARY NOT NULL PRIMARY KEY,
    expansion_json TEXT NOT NULL
) WITHOUT ROWID;
```

The key is the complete program text, compared verbatim, not a hash. Whitespace, numeric multiplicities, symbol names and the truncation order matter. There is no program normalization or symbolic-multiplicity template. The default file is `form_expansion_cache.db`, separate from the character database; the FORM module does not set a separate `user_version`.

Lookup. `get_expansion(program)` reads and decodes a hit. A miss runs `run_form`, calls `parse_form_output`, encodes the terms and commits with `ONCONFLICT(program)DONOTHING`. Failed execution or parsing does not create an entry. Each thread opens its own connection, with a PID check before reusing it, WAL, a 30-second busy timeout and up to three attempts for busy/locked writes. FORM runs outside write transactions. Concurrent identical misses can execute FORM more than once, but retain only one complete row. The context manager closes the current thread's connection. These boundaries follow the implementation and SQLite's serialized-writer model [23, 25].

Index integration. `calculate_index` and `calculate_index_internal` accept `form_cache_database_path=`; the index CLI exposes `--form-cache-database`. Otherwise the FORM database is placed beside the selected character database. A hit bypasses FORM execution and FORM-output parsing, while group-specific singlet projection still runs. Orders below two return the vacuum without external execution or cache access.

Different theories can share this expansion when their raw programs are identical and they use the same FORM database. Group and irrep meanings are supplied afterward by each theory's character basis. Their final indices can therefore differ despite a shared expansion. This reuse applies to the supported non-SU groups too; it does not require a power-sum/Schur integration backend.

Reusing lower-order character decompositions

Write the cached virtual character as

$$D_R(\mathbf{n}) = \sum_{\lambda} d_\lambda \chi_\lambda = \prod_{j\geq 1} (\psi^j \chi_R)^{n_j}, \qquad q(\mathbf{n}) = \sum_j j n_j.$$

If $\mathbf{a} + \mathbf{b} = \mathbf{n}$ componentwise, multiplication in the representation ring gives

$$D_R(\mathbf{n}) = D_R(\mathbf{a}) D_R(\mathbf{b}), \qquad d_\nu = \sum_{\lambda,\mu} a_\lambda b_\mu N^\nu_{\lambda\mu}, \quad \chi_\lambda \chi_\mu = \sum_\nu N^\nu_{\lambda\mu} \chi_\nu.$$

The identity follows by multiplying the defining Adams products; LiE's `tensor` implements the last decomposition [11, Sec. 4.5]. Signed coefficients remain valid. The two exact base cases are $D_R(\mathbf{e}_1) = \chi_R$ and $D_{\mathbf{1}}(\mathbf{n}) = 1$; neither needs a LiE call.

`CharacterDecompositionCache._decomposition_dependencies` first tries the usual split: remove one factor at the smallest occupied Adams index. For at most two factors, or when both usual dependencies are available, it retains that split. Otherwise it searches lower-weighted-order keys for the *same* Cartan type and Dynkin labels, including thread-local entries and the known first Adams operation. It considers pairs already available whose padded exponent vectors sum to the requested vector.

Among alternative pairs, the score is the product of the two numbers of irreducible terms, with zero cost for a zero or scalar-trivial factor. Ties use the total number of terms and then the vectors themselves. This estimates tensor work; it does not predict the fastest LiE call or find a globally optimal factorization. The batch dependency planner and the actual calculation use the same selection. Existing SQLite keys, schema and thread/process ownership are unchanged.

For example, if $R^{\otimes 2}$ and $(\psi^2 R)^{\otimes 2}$ are stored, their product can be obtained with one tensor operation on those entries. Atomic prerequisites need not be recalculated just to reconstruct a cached composite. On-demand generation still constructs missing prerequisites when no suitable cached split is available. It does not automatically enumerate all lower orders.

Scope. This optimization is separate from the singlet splitter. The singlet path still decomposes two partial products and contracts dual labels; the new dependency choice helps construct the required same-irrep partial products. Intermediate full tensor decompositions can remain expensive even when all individual Adams operations have already been cached.

Building every product through a chosen Adams order

`build_decomposition_cache(cartan_type,dynkin_labels,max_adams_order,...)` in `index/char_decomposition_cache.py` fills the existing decomposition table for one base irrep. At weighted order q it calls `common.math_utils.frobenius_solve(range(1,q+1),q)` to enumerate

$$\mathcal{P}_q = \left\{ (n_1, \ldots, n_q) \in \mathbb{Z}^q_{\geq 0} : \sum_{j=1}^{q} j n_j = q \right\}.$$

Each solution specifies one Adams product. This is also the multiplicity description of an integer partition: $|\mathcal{P}_q| = p(q)$, with counts $1, 2, 3, 5, 7, 11$ at orders one through six (29 rows in total).

Orders run sequentially. The builder reads existing keys at the current order, skips complete rows without decoding their payloads, and divides missing solutions into batches for a persistent `ProcessPoolExecutor` using `spawn`. Batches contain at most 64 requests, with several batches per worker to balance uneven costs. Workers read prerequisites and calculate; the parent commits each returned batch. It submits the next order only after the current one is fully committed. LiE never runs in a write transaction. Completed batches survive a later failure or interruption, so a rerun resumes from the missing entries. Independent builders may duplicate simultaneous misses; uniqueness and short transactions preserve complete stored rows.

Every composite at order q has a split into two strictly lower orders. Consequently both factors are already stored and at most one tensor operation is needed for that request, with scalar/zero shortcuts as usual. For an initially empty cache of a nontrivial irrep, one uninterrupted build needs only one individual Adams calculation per $j = 2, \ldots, M$: $M - 1$ in total. This counts logical Adams calculations, not backend retries or concurrent duplicate builds. The builder fills same-irrep decompositions, not singlet coefficients, mixed-irrep products or the FORM database.

Cutoff for the full index. A letter starts at t^2, and its j-th Adams contribution starts no earlier than t^{2j}. For each base irrep in a fixed gauge factor, its product therefore satisfies

$$\deg_t \;\geq\; 2\sum_j j n_j = 2q \quad\Longrightarrow\quad q \leq \left\lfloor \frac{N}{2} \right\rfloor \quad \text{for an index through } t^N.$$

This is a bound derived from the implemented single-letter functions. Use $M = \lfloor N/2 \rfloor$ for each required adjoint and matter irrep, retaining distinct conjugate labels. It is a sufficient bound, not a claim that every partition will occur in the projector. For product groups it holds factorwise; one bifundamental letter contributes a character to multiple factors, so one must not sum their separate orders and apply the same bound to that sum. Through t^{18}, $M = 9$ suffices. For $N < 2$ there is no cache to prebuild.

Builder API, dependencies and command-line examples

With Sage's environment active, prepare the A_2 fundamental through order nine:

```
sage -python -m index.char_decomposition_cache A2 \
  --dynkin-labels 1 0 --max-adams-order 9 --processes 4 \
  --cache-database /tmp/index-demo/char_decomposition_cache.db
```

Repeat for every required irrep. For A_2 SQCD these include $(0,1)$ and the adjoint $(1,1)$ in addition to $(1,0)$. The group rank does not multiply the maximum Adams order. The direct script entry point `sage-pythonindex/char_decomposition_cache.py...` is also supported.

```
from index.char_decomposition_cache import build_decomposition_cache

if __name__ == "__main__":
    totals = build_decomposition_cache(
        "A2", (1, 0), 9, processes=4,
        database_path="/tmp/index-demo/char_decomposition_cache.db",
        progress=lambda q, total, computed:
            print(q, total, computed),
    )
```

The return value is `dict[int,int]` mapping each order to its total product count, including reused rows. The optional callback receives `(order,total,computed)` in the parent after that order is committed. `processes=1` is serial; the default is the available CPU count. The pool starts only when there is parallel work. Use a `__main__` guard in scripts with multiple processes.

The maximum order and worker count must be positive integers. Cartan type, rank and Dynkin labels are validated. `cache_directory=` and `database_path=` are mutually exclusive. Other keywords are `lie_executable=` and `timeout=` (600 seconds per LiE call by default; Python accepts `None` for no timeout). The CLI provides `--max-order` as an alias for `--max-adams-order`, as well as `--cache-directory`, `--lie-executable`, and numeric `--timeout`. It prints computed/reused counts and exits with status 2 on a reported failure.

The builder imports `frobenius_solve` lazily and therefore needs OR-Tools only when called; ordinary index/cache lookups do not acquire that dependency. Its enumeration and complete decompositions can grow much faster than the subset used by one index. Prebuilding is optional, and a fully warm repeat still enumerates solutions and checks keys even though it runs no LiE or worker pool.

Use the prepared character database and a shared FORM database for index calls:

```
sage -python -m index.n2_theory_index theory.json --order 18 \
  --cache-database /tmp/index-demo/char_decomposition_cache.db \
  --form-cache-database /tmp/index-demo/form_expansion_cache.db
```

Without the last option, the FORM database defaults to that same directory. The corresponding index Python keyword is `form_cache_database_path=`.

Earlier sparse-projection measurements, before FORM caching

The temporary candidate was compared against commit `a9b7009` before promotion to `0a402ec`. The following historical measurements predate the FORM cache and cached-factor planner; their warm runs still execute FORM. All seven comparative index cases agreed exactly, including product groups and half hypers. There were 319 matching singlet queries; 202 also matched independent $SU(2)/SU(3)$ weight calculations. All 216 dual-label comparisons across 18 Cartan types matched Sage. All 4,996 old SQLite singlet entries tested were reused without recalculation. These are project measurements, not claims from the mathematical references.

Cold calculation	Previous (s)	Revised (s)	Speedup
A_8, through t^{18}, projection	4.599	0.849	5.42×
A_{16}, through t^{12}, projection	1.473	0.207	7.12×
A_{32}, through t^8, projection	0.469	0.117	4.01×
A_8, through t^{18}, complete index	6.213	2.443	2.54×

These are three-trial medians with fresh SQLite files and one worker for SQCD with $2(r+1)$ full fundamental hypers. Warm A8 complete-index time stayed about 1.70 s. Four-worker comparisons also agreed; small cases can incur process startup overhead. Reproduce with `test/benchmark_character_cache.py`, `--baseline-ref a9b7009`, `--baseline-label existing`, `--candidate-label sparse`.

On 10 September, $A_{32} = SU(33)$ with 66 full fundamental hypers completed through t^{18} with `timeout=None` and one worker: 94.293 s cold, 1.786 s warm. The two results matched exactly and contain 117 nonzero monomials. One LiE calculation of $\psi^3(\mathrm{adj})^2$ took 82.699 s, or 87.7% of the cold total. Each input contains 10 irrep terms; the output contains 91. FORM took 1.357 s, and all other LiE calls together took 5.066 s. This is one profiled run, not a median or an independent proof of every coefficient.

The final pairing avoids the complete tensor decomposition but not every intermediate one: $\psi^3(\mathrm{adj})^3$ splits as $\psi^3(\mathrm{adj})^2$ and $\psi^3(\mathrm{adj})$. That two-factor intermediate is the measured bottleneck. Detailed local evidence is in `output/benchmarks/singlet_projection_benchmark.md` and `output/benchmarks/a32_unlimited/report.md`.

Public result and a regression example

`calculate_index(data,order,...)` returns an exact flat Sage Laurent polynomial in (t, y, u) over $\mathbb{Q}$. `calculate_index_internal` accepts already parsed factors and hypers; `calculate_index_from_file` reads JSON, and `parse_index_polynomial` reverses the serialized polynomial format. For $SU(3)$ with six full fundamentals, the tested expansion is

$$\begin{aligned}\mathcal{I} = 1 &+ (36u^{-2} + u^4)t^4 - u^2(y + y^{-1})t^5 \\ &+ (40u^{-3} - 36 + u^6)t^6 + O(t^7).\end{aligned} \tag{3.13}$$

The displayed $O(t^7)$ is explanatory: the returned Laurent polynomial does not store a precision bound. Cache reuse improves repeated calculations, but the number of terms and tensor decompositions still grows rapidly with the cutoff.

FORM cache measurements with character projection prefilled

The controlled comparison used nine theories at t^{12} and t^{18}, one warmup and five measured repetitions per mode, with fresh clients and one worker. Both decompositions and final singlet coefficients were already filled. The character database was query-only and guards prohibited LiE execution. All 270 timed indices agreed exactly. Medians below are full calculation times in seconds, excluding setup and Python/Sage startup.

Theory, through t^{18}	Disabled	Miss/save	Hit	Speedup
$SU(2)$, 4 fundamentals	0.4323	0.4916	0.0617	7.00×
$SU(3)$, 6 fundamentals	1.7220	1.7959	0.2304	7.48×
$SU(5)$, 10 fundamentals	1.7265	1.7839	0.2648	6.52×
$SU(3)$, 1 adjoint ($\mathcal{N}=4$)	0.2808	0.3180	0.0429	6.54×
$Sp(2)$, 6 fundamentals	0.4434	0.4817	0.0634	7.00×
Spin(8), 6 vectors	0.4329	0.4848	0.0655	6.60×
G_2, 4 fundamentals	0.4418	0.4874	0.0640	6.90×
$SU(2)^2$, 2 bifundamentals	2.7421	2.7843	0.3251	8.44×
$SU(5)$, symmetric + antisymmetric	13.7565	14.1896	1.8771	7.33×

All matter in this table is full hypermultiplets; $Sp(2) = USp(4)$. Disabled mode bypasses FORM SQLite and executes/parses FORM; miss/save begins with an initialized empty FORM database and includes serialization and insertion. Hit mode decodes an existing expansion with FORM execution forbidden. The speedup is disabled time divided by hit time. At t^{12} the range was 3.23–6.92×. These are warm-filesystem measurements on this machine, not cold-disk or contention benchmarks. Evidence: `report.md`, `results.json` and `benchmark.py` under `output/benchmarks/form_expansion_theories/`.

Cross-theory hits. Two pairs share byte-identical programs: $SU(2)$ with four fundamentals and G_2 with four fundamentals; $Sp(2)$ with six fundamentals and Spin(8) with six vectors. Both directions at both orders gave eight passing reuse checks. The first theory executed FORM once; a new client for the second executed/parsed FORM zero times, decoded once, and left the database at one row. Each final index matched its own independent fresh-FORM result, while the paired final indices differed. A changed-program control missed as expected. See `cross_theory_reuse.md`, `cross_theory_reuse.json` and `check_cross_theory_reuse.py` in that same directory.

I/O cost, decomposition planning and builder verification

A separate I/O benchmark used the 113,831-term t^{18} expansion: a 589-byte program and 5,907,568-byte JSON payload (5.91 MB). Thirty measured repetitions followed three warmups, using SQLite WAL with synchronous FULL on the project filesystem. Medians are milliseconds:

Operation	Median (ms)
Serialize exact terms	179.32
SQLite insert and commit	42.07
Total save, excluding database initialization	221.51
SQLite read and fetch JSON	3.08
Reconstruct `IndexFormTerm` objects	866.42
Hit with an already open connection	870.81
Hit with a new client, including close	894.65

Initialization separately took 31.43 ms. No FORM execution/parsing is included; every round trip was checked exactly. Python reconstruction dominated loading. Individual component medians need not add to the median total. This is a single-process warm-filesystem test, not concurrent lock contention. Evidence: `output/benchmarks/form_expansion_cache_io/report.md`, `timings.json` and `benchmark.py`.

Cached-factor planner. The temporary candidate matched 130 products over A_1, A_2 fundamental/adjoint, C_2 and G_2 in serial and three-worker runs; $SU(2)$ cases also matched an independent weight oracle. Prepared-cache misses improved 1.31–1.72× for the single-request API and 1.03–1.17× for the batch API, reducing two tensor calls to one. A case with only composite entries also reduced one Adams call to zero. However, 72 full-index comparisons over six theories, with FORM prefilled and either a cold character cache or a cache filled through t^{12}, showed *no consistent overall speedup*. All indices agreed and tensor-call counts were unchanged. The retained baseline, candidate, report, scripts and raw measurements are in `output/benchmarks/adams_cache_planner/`.

Complete builder. Twelve regressions cover all 29 $SU(2)$ products through order six against independent weights, serial/parallel agreement for A_1, A_2, C_2, G_2 through order five, pool reuse, warm skips, partial-cache resume, failure persistence, input validation and both CLI entry points. A real worker log records exactly one Adams call at each of orders two through six and 23 tensor calls, with multiple worker PIDs. This verifies the operation-count claim on the tested build; no broad prebuild speedup has been measured.

The full suite rerun during the 10 September 2026 cache-documentation update ran **209 tests: 207 passed and two live-MySQL tests skipped**. It uses actual Sage/FORM/LiE for backend tests; MySQL migration and insertion unit tests use mocks unless a test server is explicitly configured. The new modules are `test_form_expansion_cache.py`, `test_cached_decomposition_planning.py` and `test_build_decomposition_cache.py`. These measurements and algorithms are source-derived project evidence, not results attributed to external papers.

3.4 Coulomb branch: invariant degrees and operator counting

MODULE `index/n2_theory_coulomb_branches.py`

The old module name `n2_theory_branches.py` has been replaced by the name above. A scalar Coulomb primary obeys $j_1 = j_2 = R = 0$ and $E = -r = \Delta$ in (3.1). Its weight is $(t^2u^2)^\Delta$. For the ordinary scalar Coulomb chiral ring, let

$$\mathcal{I}_C(x) = \sum_{\Delta \geq 0} \dim(\mathcal{R}_C)_\Delta \, x^\Delta, \qquad x = t^2u^2. \tag{3.14}$$

For a Lagrangian semisimple gauge theory, the adjoint scalar ϕ has dimension one and the invariant ring is

$$\mathbb{C}[\mathfrak{g}_\mathbb{C}]^{G_\mathbb{C}} \simeq \mathbb{C}[\mathfrak{h}_\mathbb{C}]^W = \mathbb{C}[P_1, \ldots, P_\ell], \qquad \Delta(P_i) = d_i = m_i + 1. \tag{3.15}$$

Here $\ell = \operatorname{rank} G$, d_i are Weyl invariant degrees and m_i are Weyl exponents. Hence

$$\mathcal{I}_C(x) = \mathrm{PE}\left[\sum_i x^{d_i}\right] = \prod_i \frac{1}{1 - x^{d_i}}. \tag{3.16}$$

Source: Invariant-degree interpretation and tables in Sage [14]; Casimir operators and Coulomb dimensions in Shapere–Tachikawa, §3 and §5.1 [15]; Coulomb-index letters in [16], Eq. (4.17).

Type	Basic invariant degrees (with multiplicity)
A_r	$2, 3, \ldots, r+1$
B_r, C_r	$2, 4, \ldots, 2r$
D_r	$2, 4, \ldots, 2r-2$ together with r
E_6	$2, 5, 6, 8, 9, 12$
E_7	$2, 6, 8, 10, 12, 14, 18$
E_8	$2, 8, 12, 14, 18, 20, 24, 30$
F_4	$2, 6, 8, 12$
G_2	$2, 6$

`_invariant_degrees` caches `WeylGroup(cartan_type).degrees()`. `coulomb_branch_spectrum_from_gauge_factors` accepts one `GaugeFactorData` or an iterable and returns the sorted integer tuple for all factors. Multiplicities are preserved: D_4 gives $(2, 4, 4, 6)$, not $(2, 4, 6)$. Matter information is not needed for this invariant-ring computation. The group-only function does not independently test whether matter makes the theory conformal.

3.5 Coulomb PE with rational dimensions and FORM

The spectrum interface also accepts positive exact rational dimensions, so it can serve as a counting backend for future non-Lagrangian inputs. If $f(x) = \sum_\Delta c_\Delta x^\Delta$, then

$$\mathrm{PE}[f] = \exp\left(\sum_{k\geq 1} \frac{f(x^k)}{k}\right) = \prod_\Delta (1-x^\Delta)^{-c_\Delta}. \tag{3.17}$$

The spectrum API requires $c_\Delta \in \mathbb{Z}_{\geq 0}$; the lower-level `calculate_plethystic_exponential` accepts signed integer coefficients. Negative entries give numerator factors and can describe complete-intersection relations, but an arbitrary signed list is not automatically a valid ring. Source: Plethystic exponential and product identity, [10], §5.

Clearing denominators exactly

For inclusive physical cutoff D, the implementation sets

$$L = \mathrm{lcm}(\mathrm{den}(D), \{\mathrm{den}(\Delta) : c_\Delta \neq 0,\ \Delta \leq D\}), \quad q = x^{1/L}, \quad M = LD. \tag{3.18}$$

Every input monomial becomes $q^{L\Delta}$, with integer exponent. If the lowest nonzero exponent is ν, the exact finite computation is

$$A_M(q) = \sum_{k=1}^{\lfloor M/\nu \rfloor} \frac{1}{k} \sum_\Delta c_\Delta q^{kL\Delta}, \qquad [\mathrm{PE}(f)]_{q\leq M} = \left[\sum_{j=0}^{\lfloor M/\nu \rfloor} \frac{A_M(q)^j}{j!}\right]_{q\leq M}. \tag{3.19}$$

`_build_plethystic_form_program` uses FORM's bounded q symbol and the exponential recurrence (3.9). `_parse_form_series` reads exact rational coefficients; `_to_coulomb_series` maps q^m back to $x^{m/L}$ in `PuiseuxSeriesRing(QQ,"x")`. Empty retained input returns one without starting FORM. The scaling formulas are derived directly from the source; no floating-point fit to exponents is used.

Two useful examples

For $SU(3)$, independent generators have dimensions two and three:

$$\begin{aligned}\mathcal{I}_{\mathrm{C}} &= (1-x^2)^{-1}(1-x^3)^{-1} \\ &= 1 + x^2 + x^3 + x^4 + x^5 + 2x^6 + x^7 + 2x^8 + O(x^9).\end{aligned} \tag{3.20}$$

The two terms at degree six are products P_2^3 and P_3^2; neither is a new generator. With one input dimension $6/5$ and cutoff $18/5$, the result is $1 + x^{6/5} + x^{12/5} + x^{18/5}$.

> **Mathematical input is not physical classification**
>
> Positive rational dimensions are accepted algebraically, including values that would violate the scalar unitarity bound in a unitary four-dimensional SCFT. The code does not establish that a supplied non-Lagrangian spectrum is realized by a theory or that its chiral ring is freely generated.

3.6 Taking the Coulomb limit of an existing full index

`calculate_coulomb_branch_index_from_full_index` accepts the project's Sage Laurent polynomial or its serialized string. Substitute

$$t \longrightarrow 0, \qquad u = \frac{x^{1/2}}{t}, \qquad y \text{ fixed}; \qquad t^a y^b u^c = t^{a-c} y^b x^{c/2}. \tag{3.21}$$

Thus a canonical nonzero monomial has the following disposition:

Condition	Implementation
$a > c$	Vanishes; discard it.
$a = c,\ b = 0,\ c \geq 0$	Keep its coefficient at dimension $\Delta = c/2$.
$a < c$	Raise a divergent-limit error.
$a = c,\ b \neq 0$	Reject a surviving spin-dependent term.
$a = c,\ c < 0$	Reject a negative Coulomb dimension.

An optional `max_dimension` further truncates the retained terms. Like terms are already combined in the source Laurent ring. This path is coefficient selection in Sage/Python and does not run FORM or LiE; symbolic PE/PL is where FORM is used. Source: Project-variable derivation from (3.2); in the same limit (3.3) and (3.4) become $f_V = x$, $f_H = 0$, matching [16], Eq. (4.17).

Precision cannot be created by taking a limit

If the original index is known through t^N (inclusive), then x^Δ comes from $t^{2\Delta}u^{2\Delta}$. Guaranteed Coulomb information therefore extends only to

$$\Delta \leq N/2. \tag{3.22}$$

An order-18 full index can determine Coulomb coefficients through dimension nine, not ninety. The separate index workflow obtains the order-90 Coulomb index directly from Weyl degrees instead. From (3.13), the supported Coulomb result is $1 + x^2 + x^3$ through dimension three.

> **Caller-owned cutoff and adapter restrictions**
>
> The full-index ring and the returned finite Coulomb expressions do not carry the provenance of a truncated polynomial. The caller must retain the trusted cutoff and must not interpret omitted coefficients above it as zero. The full index adapter accepts integer Laurent exponents in t, y, u; it is not a generic adapter for arbitrary fractional-power non-Lagrangian full indices. Direct Coulomb spectrum/series input supports rational dimensions more generally.

For non-Lagrangian applications this adapter also assumes the desired limit is the ordinary scalar Coulomb ring. More general protected contributions are not identified or decomposed by the code. Rejection of spin-dependent terms is a guard, not a proof of all the required physical assumptions.

3.7 Plethystic logarithm: recovering generator dimensions

For $H(0) = 1$, the inverse plethystic transform is

$$\mathrm{PL}[H](x) = \sum_{k\geq 1} \frac{\mu(k)}{k} \log H(x^k), \qquad \mathrm{PL}[\mathrm{PE}(f)] = f. \tag{3.23}$$

$\mu(1) = 1$; $\mu(k) = 0$ when k contains a squared prime factor. Otherwise $\mu(k) = (-1)^s$, where s is the number of distinct prime factors of k. Source: Plethystic logarithm/Möbius inversion, [10], §5.

`calculate_plethystic_logarithm` normalizes the input to a finite exact series, requires its constant coefficient to be one, rejects negative powers, checks any available finite-series precision, and rescales by (3.18). For input $H + O(x^P)$, the inclusive cutoff must satisfy $D < P$; a plain polynomial cannot enforce this provenance check. Write the resulting input as $1 + \delta(q)$ with lowest nonzero power ν and $K = \lfloor M/\nu \rfloor$. FORM evaluates

$$[\log(1+\delta)]_{q\leq M} = \left[\sum_{j=1}^{K} \frac{(-1)^{j+1}}{j}\delta^j\right]_{q\leq M}. \tag{3.24}$$

`_build_plethystic_log_form_program` starts with $z\delta$ and uses

$$z \longmapsto 1 - \frac{j-1}{j} z\delta, \qquad j = 2, \ldots, K. \tag{3.25}$$

After step j, this gives the completed terms up to δ^{j-1} plus $z(-1)^{j+1}\delta^j/j$; setting $z = 1$ proves the recurrence.

If FORM returns $\log H(q) = \sum_m b_m q^m$, Python then applies the exact Möbius coefficient transform

$$[q^n]\,\mathrm{PL}(H) = \sum_{k|n} \frac{\mu(k)}{k} b_{n/k}. \tag{3.26}$$

The helper names are `_mobius` and `_apply_mobius_transform`. The returned Puiseux series contains signed rational coefficients; FORM performs the ordinary-log polynomial algebra, while Python handles this arithmetic transform.

What the extraction wrapper guarantees

`extract_coulomb_branch_spectrum_from_index` expands the PL and returns a sorted tuple containing each dimension as many times as its coefficient. It requires every retained coefficient to be a nonnegative integer; otherwise it raises an error. Use the raw PL routine when signed data is required. For (3.20), the PL through dimension eight is $x^2 + x^3$ and the tuple is $(2, 3)$. For $(1 - x^4)/(1 - x^2)^2$, the PL is $2x^2 - x^4$; the raw result retains the relation and the free-spectrum wrapper rejects it.

> **A finite nonnegative PL is not a global freeness proof**
>
> Generators above the cutoff and relations first appearing later are invisible. For non-complete-intersection rings, higher PL terms can encode syzygies and their cancellations; positive terms are not universally all minimal generators. The wrapper produces a free-generator interpretation only through the supplied trusted cutoff, under the stated ring assumptions.

3.8 Coulomb API guide and reuse boundaries

All names below are in `index/n2_theory_coulomb_branches.py`.

Entry point	Input and result
`coulomb_branch_spectrum_from_gauge_factors`	One/iterable of `GaugeFactorData`; integer dimension tuple. Uses Sage degrees only.
`calculate_lagrangian_coulomb_branch_index`	Gauge-factor data and cutoff; Weyl spectrum → FORM PE → exact x series.
`calculate_coulomb_branch_index`	Exactly one of spectrum or `full_index`. Spectrum requires a cutoff; full-index mode permits it.
`calculate_coulomb_branch_index_from_full_index`	Laurent polynomial/string; select terms by (3.21).
`calculate_plethystic_exponential`	Positive rational dimensions mapped to signed integer coefficients, plus cutoff.
`calculate_plethystic_logarithm`	Unit-constant Coulomb series/string and trusted cutoff; signed rational Puiseux series.
`extract_coulomb_branch_spectrum_from_index`	Same index input; repeated `Fraction` dimensions if the retained PL is non-negative integral.
`parse_coulomb_branch_index`	Arithmetic-only serialized x expression; exact Sage Puiseux expression.

Minimal example, run with Sage Python

```
from fractions import Fraction
from anomalies.check_n2_anomalies import GaugeFactorData
from anomalies.lie_algebra import get_lie_algebra
from index import n2_theory_coulomb_branches as cb

gauge = GaugeFactorData("g", get_lie_algebra("A2"))
H = cb.calculate_lagrangian_coulomb_branch_index(gauge, 8)
dims = cb.extract_coulomb_branch_spectrum_from_index(H, 8)
assert dims == (Fraction(2), Fraction(3))

H_fractional = cb.calculate_coulomb_branch_index(
    [Fraction(6, 5)], Fraction(18, 5)
)
```

String parsers permit only the documented arithmetic variables and syntax, not general Sage code. A textual $O(x^D)$ precision annotation is not the serialized polynomial format. Supply exact fractions, not float approximations such as `1.2`, and retain the cutoff separately when serializing a truncated result.

The common property module has similarly named functions taking a *theory dictionary*, not `GaugeFactorData`. Use explicit imports to distinguish the two API layers. Its Lagrangian spectrum path reads Weyl degrees directly; it does not need to compute a PL of its already-known spectrum.

3.9 Future extension: Hall–Littlewood and Higgs operators

> **Mathematical design notes only**
> There is currently no Higgs/HL calculation module, flavor-refined index output, or Higgs-spectrum database property. This section records the mathematical background and proposed implementation changes from the session, not existing functionality.

Higgs-branch chiral primaries are scalar $\widehat{\mathcal{B}}_R$ primaries with

$$E = 2R, \qquad r = 0, \qquad j_1 = j_2 = 0. \tag{3.27}$$

The HL trace instead imposes $\delta_R = 0$ and $\delta_{1\pm} = E \pm 2j_1 - 2R - r = 0$. Solving these equations gives

$$j_1 = 0, \qquad j_2 = r, \qquad E = 2R + r, \qquad \mathcal{I}_{\mathrm{HL}} = \mathrm{Tr}_{\mathrm{HL}}(-1)^F \tau^{2(E-R)}. \tag{3.28}$$

These are conditions on contributing states, including possible fermionic cohomology representatives, not only Higgs scalar primaries. On (3.27), the exponent becomes $E = 2R$. This explains why the two sets of conditions differ without contradicting one another. Source: Gadde–Rastelli–Razamat–Yan, Eqs. (4.1), (4.5)–(4.8) and Appendix B [16]; general caveats about identifying HL with a Higgs Hilbert series in [19].

The limit in the project's variables

Keeping $\tau = t^2/u$ fixed and sending $t \to 0$ gives

$$u = t^2/\tau, \qquad t^a y^b u^c = t^{a+2c} y^b \tau^{-c}, \qquad f_H \longrightarrow \tau, \quad f_V \longrightarrow -\tau^2. \tag{3.29}$$

Thus $a + 2c = 0$ selects surviving terms, positive values vanish, and negative values signal a divergent limit. Surviving y dependence should cancel for an HL result. A full index known to t^N supports integer HL degree at most $\lfloor N/2 \rfloor$. Direct HL construction would instead use the degree-one hyper letter, so its Adams/particle bound is D, not the current full-index $\lfloor D/2 \rfloor$.

Why the vector letter represents quadratic constraints

For full hypers, the $\mathcal{N} = 2$ superpotential can be written

$$W = \sqrt{2} \sum_a \Phi_a^A \mu_a^A(Q, \widetilde{Q}), \qquad \mu_a^A = \sum_h \widetilde{Q}_h T^A_{R_{ha}} Q_h. \tag{3.30}$$

At $\Phi = 0$, the nontrivial F-term equations are $\partial W/\partial \Phi_a^A = \sqrt{2}\mu_a^A = 0$. They are quadratic in hyper scalars and transform in adj_a; derivatives with respect to the hypers vanish there. Half-hyper notation uses the invariant symplectic pairing and gives the same quadratic moment-map structure. Source: Superpotential/F-term argument in [16], Eqs. (4.9)–(4.11); general Lagrangian background in [17].

3.10 Future extension: constraints, syzygies and flavor refinement

Let $S = \mathbb{C}[Q, \widetilde{Q}]$ denote the polynomial ring of complex hyper coordinates. The Higgs chiral ring in the chosen complex structure is

$$\mathcal{R}_H = \left(S/(\mu_a^A)\right)^{G_{\mathbb{C}}}. \tag{3.31}$$

If the moment-map components form a homogeneous regular sequence, quotienting by them multiplies the equivariant Hilbert series by

$$\mathrm{PE}\left[-\tau^2 \sum_a \chi_{\mathrm{adj}_a}(g_a)\right], \qquad H_H(\tau) = \int_G d\mu\ \mathrm{PE}\left[\tau\chi_{\mathrm{hyp}} - \tau^2\chi_{\mathrm{adj}}\right]. \tag{3.32}$$

For one homogeneous non-zero-divisor f of degree d, the exact sequence $0 \to S(-d) \xrightarrow{\times f} S \to S/(f) \to 0$ gives the factor $1 - \tau^d$. Iterating is valid for a regular sequence. More generally, the fermionic factor is the Euler characteristic of the Koszul complex with odd variables η_a^A and differential $d\eta_a^A = \mu_a^A$. Its zeroth homology is the quotient; higher homology can obstruct equality with the HL index. Source: Regular/Koszul sequences [20]; physical HL/Higgs caveat [19].

For $SU(2)$, explicitly,

$$\mathrm{PE}[-\tau^2\chi_{\mathbf{3}}(a)] = (1-\tau^2a^2)(1-\tau^2)(1-\tau^2a^{-2}) = 1 - \tau^2\chi_{\mathbf{3}} + \tau^4\chi_{\mathbf{3}} - \tau^6. \tag{3.33}$$

The higher terms are the alternating exterior powers of the constraint space. One must check assumptions for the theory in question; a blanket statement that every HL index, or every genus-zero construction, equals a Higgs Hilbert series is too strong [19].

What flavor-unrefinement loses

A refined Hilbert series organizes graded operators as

$$H_H(\tau, \mathbf{a}) = \sum_{\Delta,\lambda} m_{\Delta,\lambda}\chi_\lambda^F(\mathbf{a})\tau^\Delta, \quad H_H(\tau, \mathbf{1}) = \sum_\Delta \underbrace{\sum_\lambda m_{\Delta,\lambda}\dim\lambda}_{N_\Delta}\,\tau^\Delta. \tag{3.34}$$

Unrefined data retains graded dimensions N_Δ but cannot uniquely recover flavor representations. It is sufficient if only those counts or an unrefined PL are desired, subject to the same Higgs/HL and syzygy caveats. Highest-weight generating functions are a possible later representation-level summary [21].

Proposed implementation changes

Preserve flavor characters in the letter builder and under every Adams operation. Complex blocks use $R \otimes \mathbf{n}$ and $\overline{R} \otimes \overline{\mathbf{n}}$; real blocks use the $Sp(n)$ defining representation; pseudoreal blocks use the $SO(m)$ vector. Project only gauge factors, leaving flavor characters in the output. Simple-factor LiE/cache machinery can be reused, but the coefficient ring, parsers and serialization must support flavor labels and explicit abelian flavor charges. A refined PL must substitute *all* flavor variables. Share flavor-block construction in a lower-level helper to avoid a circular import between properties and index.

4 Package `common`

4.1 Flavor symmetry and exactly marginal gauge couplings

MODULE `common/n2_theory_properties.py`

The property layer reuses anomaly-validated matter and groups it into flavor blocks. The symmetry is the commutant of the gauge action on the hyper space, with its compatible reality structure. At zero masses the connected factors are

$$F_{\mathcal{R}} = \begin{cases} U(n), & \mathcal{R} \text{ complex, } n \text{ full hypers,} \\ Sp(n), & \mathcal{R} \text{ real, } n \text{ full hypers,} \\ SO(m), & \mathcal{R} \text{ pseudoreal, } m = 2n_{\text{full}} + n_{\text{half}}. \end{cases} \tag{4.1}$$

Here $Sp(n)$ has rank n and defining dimension $2n$, often written $USp(2n)$. Complex-conjugate labels are merged into a single block, including simultaneous conjugation of all product factors. A pseudoreal block counts half-hyper units. An $SO(1)$ block has no continuous flavor symmetry. Source: Hyper reality in [3]; flavor/commutant background in [17]; the explicit grouping rule is the project's implementation of that construction.

The helpers `_simple_flavor_blocks`, `_product_flavor_blocks`, `_canonical_complex_key` and `_calculate_flavor_symmetry` carry out this grouping. This is flavor-symmetry metadata only: it is not used to refine the current index. Disconnected components, faithful global quotients and strong-coupling accidental enhancements are not determined by this routine.

For an anomaly-free conformal Lagrangian theory with s simple factors, the code assigns one exactly marginal complex gauge coupling per factor:

$$\tau_a = \frac{\theta_a}{2\pi} + \frac{4\pi i}{g_a^2}, \qquad \dim_{\mathbb{C}} \mathcal{M}_{\text{conf}} = s. \tag{4.2}$$

This is the local dimension of the Lagrangian conformal manifold, not a description of its global S-duality quotient or singular loci. The τ_a in this equation is a coupling, distinct from the HL grading τ. Source: Lagrangian conformality [3]; $\mathcal{N} = 2$ supersymmetry-preserving marginal deformations in Córdova–Dumitrescu–Intriligator, §3.2.2/Table 22 [18].

Examples

$SU(3)$ with six full fundamental hypers has a $U(6)$ flavor block and one exactly marginal gauge coupling. $SU(2)$ with four full fundamentals is instead a pseudoreal block with eight half-hyper units, hence $SO(8)$ flavor. The distinction follows from total representation reality, not just multiplicity.

4.2 Central charges and property orchestration

For a conformal Lagrangian realization, define the effective numbers of vector and hypermultiplets by representation dimensions:

$$n_v = \sum_a \dim \mathfrak{g}_a, \qquad n_H = \sum_h n_h \epsilon_h \prod_a \dim R_{ha}. \tag{4.3}$$

The Weyl-anomaly coefficients are

$$a = \frac{5n_v + n_H}{24}, \qquad c = \frac{2n_v + n_H}{12}. \tag{4.4}$$

These include free gauge-singlet hypers. The code checks positivity and the equivalent identities $4(2a - c) = n_v$, $4(5c - 4a) = n_H$. Source: Shapere–Tachikawa, free-multiplet values in Eqs. (2.5)–(2.6) [15].

An additional mathematical cross-check with the Lagrangian Coulomb spectrum is

$$4(2a - c) = \sum_i (2d_i - 1) = \dim \mathfrak{g}. \tag{4.5}$$

The final equality follows from $\sum_i (d_i - 1) = |\Phi^+|$ and $\dim \mathfrak{g} = \ell + 2|\Phi^+|$. The more general Coulomb-dimension relation requires the assumptions discussed in [15], §5.1, Eq. (5.4); the project does not infer arbitrary non-Lagrangian central charges from it. The cross-check with degrees is explanatory here, not a separate implemented validation step. For $SU(3)$ with six full fundamentals, $n_v = 8, n_H = 18$, so $a = 29/12$, $c = 17/6$ and $4(2a - c) = 3 + 5 = 8$.

Basic properties without index calculation

`calculate_n2_theory_properties(data)` validates the input and returns flavor metadata, gauge marginal couplings, conformal-manifold dimension and exact central charges. It never computes a full index, a Coulomb index or the Coulomb spectrum; those keys are omitted from the basic dictionary.

Basic property key	Representation
`group`	Gauge-group display name.
`lagrangian_scft_candidate`	Combined SCFT-candidate flag.
`flavor_symmetry`	Flavor-factor metadata.
`conformal_manifold_dimension`	Number of marginal gauge couplings.
`exactly_marginal_gauge_couplings`	Gauge-factor IDs.
`central_charges`	Exact rational a, c only.

For valid non-candidates, flavor metadata remains available; central charges and conformal-manifold dimension are `None`, with no marginal gauge couplings. Malformed input raises an error. The basic calculator is descriptive; the database additionally rejects a theory that fails anomaly or conformality checks before inserting its theory data.

Separate index-related properties

`calculate_n2_theory_indices(data,order=18,max_dimension=90)` validates once, then computes the full index, the gauge-degree Coulomb index and the complete spectrum. Full order N and Coulomb dimension D are independent, inclusive cutoffs. Omitting either argument uses the corresponding module default. Valid non-candidates receive `None` for all five fields.

Index-related key	Representation
`superconformal_index`	Serialized flat Laurent polynomial string.
`superconformal_index_order`	Nonnegative integer N; default 18.
`coulomb_branch_index`	Serialized x-series expression string.
`coulomb_branch_index_max_dimension`	Exact nonnegative rational D; default 90.
`coulomb_branch_spectrum`	Sorted tuple of exact `Fraction` dimensions, with multiplicities; independent of N, D.

The individual `calculate_superconformal_index(data,order=...)` and `calculate_coulomb_branch_index(data,max_dimension=...)` wrappers retain raw Sage results. `calculate_coulomb_branch_spectrum(data)` returns the complete tuple. The calculator does not write to MySQL or decide whether a stored result should be replaced; the database API applies that policy.

The property CLI defaults to basic properties. Select the second stage with `--indices`, optionally `--index-order` and `--coulomb-max-dimension`; supplying cutoffs without `--indices` is rejected. `calculate_n2_theory_properties_from_file` also stays basic.

4.3 Shared exact arithmetic, FORM execution and JSON

MODULE `common/number_utils.py; common/form_utils.py; common/json_utils.py`

These modules hold common utilities formerly duplicated in physics modules. Their role is to keep exact arithmetic and external-process handling consistent.

Exact-number contracts

Routine	Validation
`as_integer`	Rejects Python booleans/floats, including `1.0`; requires exact equality with the converted integer.
`as_nonnegative_int`	Calls `as_integer`, then requires $n \geq 0$.
`as_nonnegative_fraction`	Accepts supported exact rational objects or integer/fraction text; rejects Python booleans/floats and negative values.
`as_positive_fraction`	Adds the strict $q > 0$ condition.

Use `Fraction(6,5)` or Sage `QQ(6)/5` for a Coulomb dimension; use integer counts for Dynkin labels, hyper multiplicities and cutoffs in t. The rational-text format is an integer or fraction such as `"6/5"`, not decimal text such as `"1.2"`. Accepting integral floats would broaden the shared API everywhere, not just in the Coulomb module; the current behavior is deliberately strict. These validators are type/value checks, not physical unitarity or representation-existence proofs.

One FORM execution boundary

`run_form(program,form_executable=...,timeout=...)` writes a FORM program in an isolated temporary directory, invokes the executable with quiet output, and returns the output text. It turns missing executables, timeouts, nonzero exit status or reported stderr into errors with useful context. Both the full index and Coulomb PE/PL reuse this function; there is no need for separate subprocess wrappers in each module.

`split_top_level` and `split_signed_terms` support parsing expressions without splitting inside parentheses or mistaking an exponent sign for a new term. Physics-specific parsers remain with their own modules because character factors and Puiseux powers require different validation. Source: Execution and parser contracts from the source; FORM program syntax in [12].

Serialization is exact, but precision metadata is separate

The JSON helpers encode a fraction as

```
{"numerator": 6, "denominator": 5}
```

Tuples become arrays, dataclass records become structured objects, and the separate index-property layer serializes Sage indices as expression strings. For example, the spectrum $(2, 3)$ is stored as an array of the two exact fractions, not as the polynomial $x^2 + x^3$.

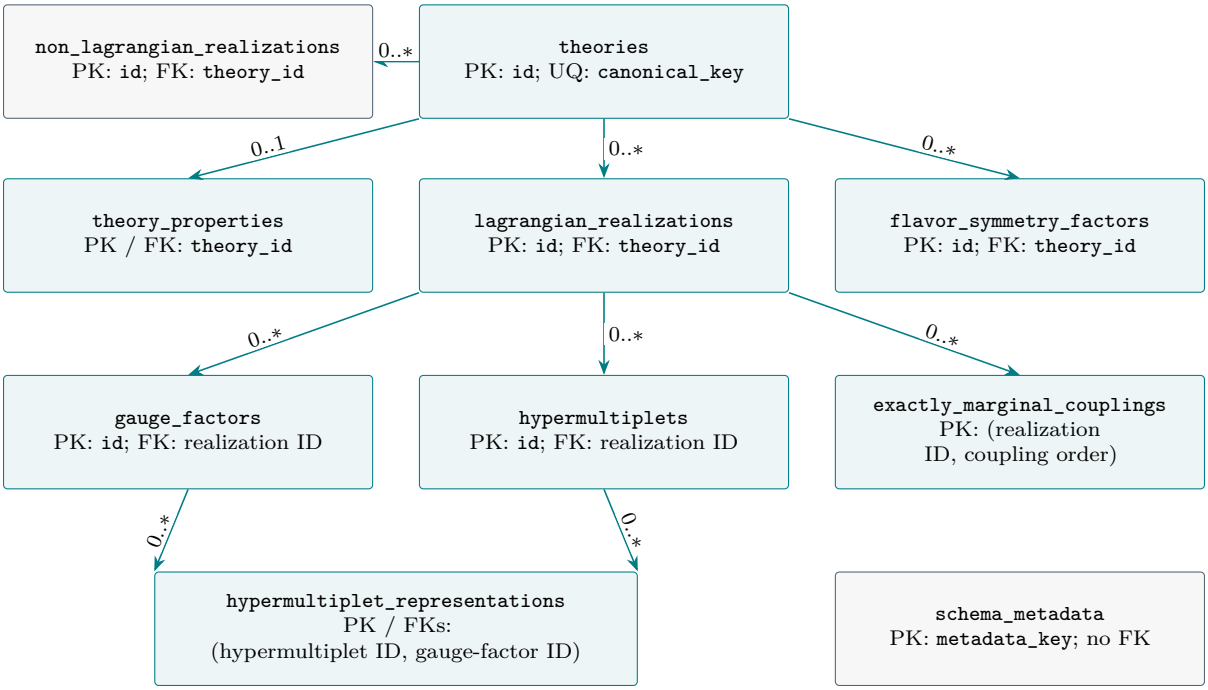
Index strings do not embed their trusted truncation order. The separate calculator returns it explicitly, and MySQL schema 7 stores the two cutoffs beside the expressions and in combined JSON. The last nonzero term is not a substitute for this metadata. Future non-Lagrangian imports still need their own validated input/provenance workflow; that importer is not implemented.

4.4 MySQL persistence and schema version 7

MODULE `common/n2_theory_db.py`

The database distinguishes a theory, its shared properties, and its realizations. Flavor factors belong to the theory; gauge factors, hypermultiplets and exactly marginal couplings belong to a Lagrangian realization. The non-Lagrangian table is a placeholder, with no implemented import workflow.

Database relationships



Arrows run from parent to child; labels give the number of children per parent allowed by the schema. Each child has exactly one parent along each foreign key. PK = primary key; FK = foreign key; UQ = unique key. "Realization ID" abbreviates `lagrangian_realization_id`. All declared FKs use `ON DELETE CASCADE`.

How product-group matter is represented

One `hypermultiplets` row holds a complete tensor-product representation and its submitted `kind` and `multiplicity`. Its rows in `hypermultiplet_representations` identify the representation under each gauge factor, including singlets. The latter table joins matter to gauge factors; it does not create a separate hypermultiplet for each factor.

> **Shared flavor versus the submitted matter split**
>
> Schema 6 retains `half_hyper_units` in the shared flavor table and JSON, with $m = 2n_{\text{full}} + n_{\text{half}}$. The original split remains in realization rows and input/anomaly JSON. Public property-calculator output still reports both counts; the database strips them from its shared-property copy.

4.5 Database table catalog and keys

All tables use InnoDB and `utf8mb4`. IDs are unsigned `BIGINT`s; standalone `id` keys auto-increment. Exact rational values use JSON numerator/denominator records or separate numeric columns, as detailed below.

Table	Keys and stored data
`schema_metadata`	PK `metadata_key`; `metadata_value` holds the schema version (current value `"7"`). Independent of theory rows.
`theories`	PK `id`; UQ `canonical_key`. Display name and creation/update timestamps. A new Lagrangian theory uses `lagrangian:<hash>` as its key.
`theory_properties`	PK/FK `theory_id` → `theories.id`. Flavor group/rank/dimension, conformal-manifold dimension, exact central-charge JSON, separate full-index, Coulomb-index and Coulomb-spectrum JSON, two nullable cutoff columns, and combined `properties_json`. Generated, indexed `DECIMAL(65,30)` columns approximate a, c for queries.
`flavor_symmetry_factors`	PK `id`; FK `theory_id` → `theories.id`; UQ (`theory_id`, `factor_order`). Group, Lie algebra/rank/dimension, representation reality, `half_hyper_units`, and `gauge_representation_json`. No full/half split columns since schema 6.
`lagrangian_realizations`	PK `id`; FK `theory_id` → `theories.id`; UQ `canonical_hash` (64-character SHA-256). Gauge group/count, anomaly and conformality flags, input/anomaly/coupling JSON, creation timestamp.
`non_lagrangian_realizations`	PK `id`; FK `theory_id` → `theories.id`. Construction type, optional source reference, `data_json`, creation timestamp. Reserved.
`gauge_factors`	PK `id`; FK realization ID → `lagrangian_realizations.id`. Separate UQs on (realization ID, `factor_order`) and (realization ID, `factor_key`). Group, Cartan type/rank, exact vector, matter and b_0 fractions, beta/global-anomaly flags and Witten parity.
`hypermultiplets`	PK `id`; FK realization ID → `lagrangian_realizations.id`; UQ (realization ID, `hyper_order`). Name, `kind` (full/half), submitted multiplicity, total tensor-product dimension and reality.
`hypermultiplet_representations`	Composite PK (`hypermultiplet_id`, `gauge_factor_id`); separate FKs to `hypermultiplets.id` and `gauge_factors.id`. Name, Dynkin-label JSON, dimension/reality, exact Dynkin-index and beta-contribution numerators/denominators.
`exactly_marginal_couplings`	Composite PK (realization ID, `coupling_order`); FK realization ID → `lagrangian_realizations.id`. Stores `gauge_factor_key` as text; that key has no additional FK to `gauge_factors`.

Serialization and integrity boundaries

Full and Coulomb indices are JSON expression strings. Coulomb spectra are arrays of exact fractions; central charges are exact a, c fraction records. The separate computed fields and their cutoffs are mirrored into `properties_json` when filled. Initial imports leave computed columns NULL and omit those keys from the basic JSON. Gauge beta fractions and representation invariants use `DECIMAL(65,0)` integer columns. The writer keeps both ends of each matter–gauge relation within the same realization; the two individual FKs do not themselves enforce that match.

4.6 Canonical matter, migrations and import behavior

The canonical Lagrangian hash is SHA-256 of a normalized payload. Names resolve to Dynkin labels, repeated matter entries combine, and zero multiplicities vanish. Conjugate choices use the lexicographically larger label tuple, favoring fundamentals. Product representations identify only simultaneous conjugation of all factors. Matter order is ignored; gauge-factor order remains significant.

Full/half equivalence for pseudoreal representations

For each complete pseudoreal gauge representation, aggregate before pairing:

$$m = 2n_{\text{full}} + n_{\text{half}}, \qquad (n_{\text{full}}^{\text{canonical}}, n_{\text{half}}^{\text{canonical}}) = (\lfloor m/2 \rfloor,\ m \bmod 2).$$

Four full $SU(2)$ fundamentals, eight halves, or two full plus four halves now have the same hash. An odd remaining half stays in its own irrep. For products, the rule uses the reality of the entire tensor product. Real/complex matter remains full; invalid half-hyper input is rejected by the checker.

Schema history

Migration	Change
$1 \to 2$	Add generated, indexed decimal central charges; exact JSON remains authoritative.
$2 \to 3$	Rename the plural full-index property and column to singular `superconformal_index` / `superconformal_index_json`.
$3 \to 4$	Rename the old Coulomb-spectrum/index placeholder to `coulomb_branch_index_json`, including its combined-JSON key.
$4 \to 5$	Add nullable `coulomb_branch_spectrum_json` for actual generator dimensions; do not compute spectra for existing rows.
$5 \to 6$	Drop `full_hypermultiplets` and `half_hypermultiplets` from `flavor_symmetry_factors`; keep `half_hyper_units` and the original realization records.
$6 \to 7$	Add nullable full-index order and exact Coulomb-cutoff JSON; preserve existing results and leave unrecorded legacy precision unknown.

> **Schema upgrade and data migration are different operations**
>
> The schema-6 and schema-7 migrations do not rehash realizations, merge duplicate theories, or eagerly rewrite old shared JSON. Old hashes from the earlier conjugacy or full/half conventions require a separate data migration. Under the current conjugacy convention, inputs already written entirely as full hypers keep their hashes after full/half normalization.

`_insert_shared_properties` compares only basic physical properties, ignoring the obsolete flavor split in legacy JSON. Matching JSON is normalized without erasing existing index data. Spectrum filling now belongs to the separate worker. Physical mismatches still reject attachment; shared flavor metadata includes gauge labels and factor IDs, so this is not a general comparison of arbitrary dual realizations.

`store_lagrangian_theory` checks anomalies/conformality and calculates basic properties before looking up the hash. It performs no index or spectrum calculation. A duplicate returns the existing IDs with `inserted=False` and does not change the name, `updated_at`, realization or shared data. A new import uses parameterized SQL and one explicit data transaction. Schema initialization precedes that transaction. There is no automatic recognition of dual descriptions; attachment with `theory_id` still requires matching basic shared properties.

4.7 Independent precision and replacement rules

The database stores one set of physical properties per theory. A Lagrangian realization supplies the input for later calculation; different realizations attached to the same theory share its indices and Coulomb spectrum. Initial imports run the anomaly/conformality checks and calculate basic properties only. Both indices, the spectrum and both precision columns start as SQL `NULL`; their keys are initially absent from `properties_json`.

The separate calculator returns full-index precision $N \in \mathbb{Z}_{\geq 0}$ and Coulomb precision $D \in \mathbb{Q}_{\geq 0}$. These are inclusive cutoffs:

$$\mathcal{I}_{\leq N} = \sum_{a=0}^{N} t^a P_a(y,u), \qquad \mathcal{I}_{\mathrm{C},\leq D} = \sum_{0\leq\Delta\leq D} c_\Delta x^\Delta.$$

A zero boundary coefficient does not reduce the trusted cutoff. For example, $1 + x^2 + x^4$ computed through $D = 5$ must retain precision five, even though its largest nonzero exponent is four. The complete gauge-invariant generator spectrum is independent of either cutoff; it is not truncated with the index.

For each of the two indices separately, let V be its stored value and $p_{\mathrm{old}}, p_{\mathrm{new}}$ its stored and submitted cutoffs. The write rule is

$$\text{replace or fill} \iff V \text{ is missing} \quad \text{or} \quad \big(p_{\mathrm{old}} \text{ is known and } p_{\mathrm{new}} > p_{\mathrm{old}}\big).$$

Stored / submitted state	Database action
Stored index missing	Fill it and record the supplied cutoff.
Known, strictly higher cutoff	Replace that index and its cutoff.
Equal or lower cutoff	Keep the existing string and precision unchanged.
Legacy index, unknown cutoff	Keep it; report `unknown_precision`.
Omitted or `None` component	Leave the corresponding stored data alone.
Supplied index without a cutoff	Reject the payload before opening the write transaction.

For example, an update from $(N, D) = (18, 90)$ to $(24, 60)$ upgrades the full index only. The Coulomb index remains at dimension 90. Equal-order submissions do not replace even a different string; this API is not a correction/force-update mechanism. The caller supplies the trusted calculation cutoff; the writer does not infer or prove precision from the expression's last monomial.

A missing spectrum is filled. An identical complete spectrum is retained, with multiplicities preserved; a conflicting spectrum raises an error and rolls back that update call. This policy differs from comparison of truncated indices.

Implementation source: `common/n2_theory_properties.py`, `common/n2_theory_db.py`; the inequalities above express the implemented cutoff contract, not an additional physical conjecture.

4.8 Schema 7, legacy data and atomic writes

Schema 7 keeps the same ten tables and adds two nullable columns to `theory_properties`. Central charges and all existing index data are preserved by the migration.

Column / result key	Stored representation
`superconformal_index_order`	`BIGINT UNSIGNED`; inclusive integer N.
`coulomb_branch_index_max_dimension_json`	Exact numerator/denominator JSON for D.
`coulomb_branch_index_max_dimension`	Corresponding key in calculator output and combined JSON.

The three computed values still use `superconformal_index_json`, `coulomb_branch_index_json` and `coulomb_branch_spectrum_json`. Successful updates mirror changed values and cutoffs into `properties_json` while preserving basic properties and other computed components.

Migration $6 \to 7$ adds the columns without guessing legacy cutoffs, recomputing indices, rehashing realizations or merging theories. Initialization applies migrations when the database is next opened through the database API. MySQL schema DDL precedes the explicit data transaction and is not rolled back with it.

When an old result's original requested cutoff is independently known, record it before normal upgrades:

```
from common.n2_theory_db import record_lagrangian_index_cutoffs

# Use these numbers only if they are the verified original cutoffs.
record_lagrangian_index_cutoffs(
    connection, realization_id, order=18, max_dimension=90,
)
```

This function fills unknown precision only. It rejects changing an already known cutoff or annotating an index that is absent. It changes neither index expression. Merely seeing a last term of degree 18 does not establish that the original calculation was requested through 18.

`update_lagrangian_indices(connection,realization_id,indices)` accepts `calculate_n2_theory_indices`'s output or a subset of computed components with their matching cutoffs. It opens a short transaction, locks the shared row with `SELECT ... FOR UPDATE`, rechecks the current cutoffs, and merges accepted changes into the columns and JSON together. Failure rolls back the call; a no-op does not change the theory's `updated_at` timestamp.

Calculations run before this lock is acquired. Thus a slower low-order worker cannot overwrite a higher-order result committed while it was calculating. Use an initialized connection with no caller-owned transaction for update and legacy-cutoff APIs. These APIs manage their own commit/rollback boundary.

4.9 Separate worker, partial retries and reproducible use

The program `common/n2_theory_db_indices.py` streams stored Lagrangian inputs in bounded pages, using the smallest realization ID for each theory. It visits each shared theory once rather than recalculating the same shared properties for every attached realization.

1. Default mode selects any missing full index, Coulomb index or spectrum. SQL `NULL` and JSON `null` are both treated as missing.
2. `--upgrade` additionally selects indices with known lower cutoffs. Already sufficient components are not calculated. Complete legacy records with unknown precision are reported without recalculating those indices.
3. Each requested component is calculated outside a database transaction, then submitted to the locked update API. The stored state is checked again at write time, even if it differed when the job was selected.
4. Each successful component commits independently. A failure is reported for that component; subsequent components and theories continue. Re-running retries remaining missing or lower-order components and keeps earlier successes.

Multiple workers can duplicate a calculation; there is no persistent job-claim queue. The row-lock and cutoff policy protects data against downgrades, not against duplicated CPU work. A conflicting spectrum rolls back its own update call, not components already saved by earlier worker calls.

Run from the repository root with Sage's Python. Database credentials follow the existing `N2_DB_HOST`, `N2_DB_USER`, `N2_DB_PASSWORD` and `N2_DB_UNIX_SOCKET` environment conventions; port is a CLI option.

```
# Stage 1: validate and store basic properties.
sage -python -m common.n2_theory_db \
  anomalies/example_e6.json DATABASE --user USER
# Stage 2: fill missing components, defaults N=18 and D=90.
sage -python -m common.n2_theory_db_indices DATABASE --user USER
# Later: upgrade only components with lower known precision.
sage -python -m common.n2_theory_db_indices DATABASE --user USER \
  --upgrade --index-order 24 --coulomb-max-dimension 100 --limit 10
```

The worker also accepts `--host`, `--port`, `--unix-socket`, `--connect-timeout`, `--cache-directory`, `--lie-executable`, `--form-executable`, `--timeout` and `--processes`. `--limit` bounds visited theory jobs, including reported legacy skips. The API `fill_lagrangian_indices` yields the same per-theory outcomes. Each outcome reports `updated_fields`, `skipped_fields` and `errors`; a final CLI summary counts processed and failed jobs. Exit codes are 0 for no component failures, 1 for component failures, and 2 for an outer configuration or database-level error.

Validation recorded during implementation, 12 September 2026. All 238 backend tests passed in 21.920 seconds, including an isolated MySQL 8.0.46 server, schema migration, concurrent upgrades, rollback, partial retries, JSON-null selection and real Sage/FORM/LiE calculations. This PDF update checks source contracts and document rendering; it does not rerun that suite or alter the user's database. Older cache-performance measurements remain dated evidence.

5 Package `test` and reproducible use

Selected test modules	Main coverage
`test_check_n2_anomalies.py`	Root/representation data, all families, spectator factors, full/half validation, beta and Witten checks.
`test_n2_theory_index.py`	Laurent output, named/explicit labels, product singlets, low-order reference indices, LiE parsing and cache concurrency.
`test_n2_theory_coulomb_branches.py`	Weyl degrees, repeated/fractional dimensions, PE/PL round trips, relations, full-index projection and precision.
`test_n2_theory_properties.py`	Flavor blocks, central charges, property separation, cutoff metadata, direct Coulomb wiring and exact JSON.
`test_n2_theory_db.py`	Schema/version handling, shared property comparison, full/half deduplication, preservation of existing indices, SQL parameters and transactions; actual index equality through t^6.
`test_n2_theory_db_indices.py`	Deferred worker, schema-7 migration, independent precision, legacy metadata, concurrent updates, partial retries, rollback, JSON-null selection and real index storage.
`test_form_expansion_cache.py`	Exact term serialization, raw-program keys, thread/process safety, failure behavior and index integration.
`test_cached_decomposition_planning.py`	Reuse of cached composite blocks, base identities and consistent serial/parallel dependency planning.
`test_build_decomposition_cache.py`	Complete enumeration, minimal Adams calls, parallel batches, resume and CLI contracts.

5.1 Validation snapshot

The two-stage database implementation was validated on 12 September 2026: **238 tests passed in 21.920 s, with no skips**. The run used actual Sage/FORM/LiE and a disposable MySQL 8.0.46 server. Migration, race protection, rollback, partial retries and real stored indices were exercised without altering the user's database. This is recorded implementation evidence; the present PDF edit rebuilds and visually checks documents, not the test suite.

The 10 September cache-documentation rerun was 209 tests in 19.658 s: 207 passed and two MySQL tests were skipped. Earlier 134- and 170-test figures also describe older snapshots. Keep those dated measurements distinct from the current 238-test validation; cache benchmarks were not rerun here.

5.2 Commands and dependencies

Run from the repository root with Sage's Python, not an unrelated Python interpreter. FORM and LiE must be discoverable for backend tests.

```
sage -python -m unittest discover -s test -p 'test_*.py' -v
sage -python -m index.n2_theory_index --help
sage -python -m index.char_decomposition_cache --help
sage -python -m common.n2_theory_properties --help
sage -python -m common.n2_theory_db --help
sage -python -m common.n2_theory_db_indices --help
```

Sage supplies exact rings and Lie data; FORM and LiE handle expansions and representation decompositions. PyMySQL supplies the database connection. The low-level APIs accept executable paths, timeouts and cache paths. The complete Adams-cache builder lazily imports the OR-Tools-based Frobenius solver; ordinary index/cache lookups do not require OR-Tools.

5.3 Practical boundaries before extending the project

Preserve the exact-number contracts, distinct index/spectrum properties and source precision. Non-Lagrangian counting needs explicit ring/freeness assumptions; Higgs work needs an HL/Hilbert-series justification; flavor refinement must preserve characters through gauge projection. These are mathematical contracts.

References and equation provenance

Equation numbers in this document are local. Citations near equations identify the primary source or software reference; algebraic recurrences, cutoff rules and function mappings are derived from the inspected project source. Literature section/equation numbers refer to the linked arXiv editions where specified. Base software documentation was consulted on 8 September 2026; the new orthogonality, Sage duality and SQLite references were checked on 10 September. This bibliography also retains the relevant sources from both earlier summaries.

References

[1] The Sage Developers, *Root systems*, SageMath reference manual. Dynkin labels, root/weight lattices and finite Cartan data. Official root-system documentation.

[2] The Sage Developers, *Weyl character rings*, SageMath reference manual. Character degrees, duals and Frobenius–Schur indicators. Official Weyl-character documentation.

[3] L. Bhardwaj and Y. Tachikawa, *Classification of 4d $\mathcal{N} = 2$ gauge theories*, JHEP 12 (2013) 100, updated arXiv edition. arXiv:1309.5160. §2, Eqs. (2.1)–(2.8): matter, spectator dimensions and beta functions.

[4] R. Feger, T. W. Kephart and R. J. Saskowski, *LieART 2.0 – A Mathematica Application for Lie Algebras and Representation Theory*, Comput. Phys. Commun. 257 (2020) 107490. arXiv:1912.10969. Representation tables are cross-checks; LieART is not the project backend.

[5] A. Bilal, *Lectures on Anomalies*. arXiv:0802.0634. Chiral anomaly traces, conjugation and product-group anomaly structure.

[6] E. Witten, *An $SU(2)$ Anomaly*, Phys. Lett. B 117 (1982) 324–328. doi:10.1016/0370-2693(82)90728-6. Conventional four-dimensional mod-two gauge anomaly.

[7] J. Wang, X.-G. Wen and E. Witten, *A New $SU(2)$ Anomaly*, J. Math. Phys. 60 (2019) 052301. arXiv:1810.00844. §2.3 includes the conventional isospin parity formula and distinguishes the additional non-spin anomaly.

[8] I. García-Etxebarria and M. Montero, *Dai–Freed anomalies in particle physics*, JHEP 08 (2019) 003. arXiv:1808.00009. Global-anomaly scope beyond local anomaly polynomials.

[9] A. Gadde, E. Pomoni and L. Rastelli, *The Veneziano Limit of $\mathcal{N} = 2$ Superconformal QCD: Towards the String Dual of $\mathcal{N} = 2$ $SU(N_c)$ SYM with $N_f = 2N_c$*, arXiv:0912.4918. §5, Eqs. (5.4)–(5.5), (5.21)–(5.22): the right-index convention and letters.

[10] S. Benvenuti, B. Feng, A. Hanany and Y.-H. He, *Counting BPS Operators in Gauge Theories: Quivers, Syzygies and Plethystics*, JHEP 11 (2007) 050. arXiv:hep-th/0608050. §5: PE, inverse plethystics, complete intersections and refined counting.

[11] M. A. A. van Leeuwen, A. M. Cohen and B. Lisser, *LiE: A Package for Lie Group Computations* (1992), and LiE reference manual, character operations (Adams and tensor products). TU Eindhoven publication record; CWI manual record.

[12] J. A. M. Vermaseren, J. Davies, T. Kaneko, J. Kuipers, C. Marinissen, B. Ruijl, M. Tentyukov, T. Ueda and J. Vollinga, *FORM version 5.0.0 Reference manual*, 27 January 2026. User-supplied `form-5.0.0-manual.pdf`; see Chapters 1–3 and the entries for symbol restrictions, substitution, sorting and rational functions.

[13] J. Kuipers, T. Ueda, J. A. M. Vermaseren and J. Vollinga, *FORM version 4.0*, Comput. Phys. Commun. 184 (2013) 1453–1467. arXiv:1203.6543. Software background; FORM 5 syntax is taken from the supplied manual.

[14] The Sage Developers, *Finite Coxeter groups*, `degrees()` documentation. Definition as fundamental invariant degrees, including product groups and the repeated D_4 degree. Official finite-Coxeter-group documentation.

[15] A. D. Shapere and Y. Tachikawa, *Central charges of $\mathcal{N} = 2$ superconformal field theories in four dimensions*, JHEP 09 (2008) 109. arXiv:0804.1957. Eqs. (2.5)–(2.6): free multiplets; §3 and Eq. (5.4): Casimir dimensions and $4(2a - c) = \sum_i (2\Delta_i - 1)$, with stated assumptions.

[16] A. Gadde, L. Rastelli, S. S. Razamat and W. Yan, *Gauge Theories and Macdonald Polynomials*, Commun. Math. Phys. 319 (2013) 147–193. arXiv:1110.3740. §4, Eqs. (4.5)–(4.11) and (4.15)–(4.17), Appendix B: HL and Coulomb limits.

[17] Y. Tachikawa, $\mathcal{N} = 2$ *supersymmetric dynamics for pedestrians*, Lecture Notes in Physics 890 (2014). arXiv:1312.2684. Lagrangian, hypermultiplet, flavor and moduli-space background.

[18] C. Córdova, T. T. Dumitrescu and K. Intriligator, *Deformations of Superconformal Theories*, JHEP 11 (2016) 135. arXiv:1602.01217. §3.2.2/Table 22: four-dimensional $\mathcal{N} = 2$ preserving deformations.

[19] M. J. Kang, C. Lawrie, K.-H. Lee, M. Sacchi and J. Song, *Higgs, Coulomb, and Hall–Littlewood*, Phys. Rev. D 106 (2022) 106021. arXiv:2207.05764. Examples and caveats showing that the HL index and Higgs Hilbert series need not coincide, including twisted class-S constructions.

[20] The Stacks Project Authors, *Koszul regular sequences*, Tag 062D. Regular sequences, Koszul homology and exactness criteria used in the constraint-counting derivation.

[21] A. Hanany and R. Kalveks, *Highest Weight Generating Functions for Hilbert Series*, JHEP 10 (2014) 152. arXiv:1408.4690. Representation-level organization of refined Hilbert series.

[22] P. Etingof, *Lie Groups and Lie Algebras*, MIT 18.755 lecture notes (2024), Proposition 32.1 (dual highest weights) and Corollary 35.9 (character orthogonality). Official MIT lecture notes.

[23] SQLite, *Write-Ahead Logging*, especially §2.2 Concurrency. Official WAL documentation.

[24] SQLite, *Datatypes In SQLite*, §2 Storage Classes and Datatypes. Official datatype documentation.

[25] Python Software Foundation, *sqlite3: DB-API 2.0 interface for SQLite databases*; connections, transactions and thread checks. Official Python documentation.